
Week 4. Homework 2. SQL-II

Submission Guidelines

- **Dataset:** You must use the provided *IMDB.sqlite* dataset to answer all questions.
- **Code and Output:** Copy your code and attach screenshots of your outputs under each question.
- **Submission Format:** Each student is required to push one Word or PDF file to a designated GitHub repository you create specifically for this week's assignment.
- **Repository Name:** Use the format *Week 4-HW2-LastName*, where *LastName* is your last name.
- **Code Quality:** Ensure your code is organized for easy understanding.
- **AI Usage:** You should make a serious attempt at each problem prior to using an LLM. Remember, you may be tested on SQL knowledge without assistance in a technical interview. If prompting an LLM, I suggest telling it your starting point, your thought process, and asking it to guide you. I've done this learning SQL and it is helpful and I learned more that way. Disclose your usage at the end of the file; you will not be penalized regardless, but this give me some information.

Questions

Date-Based Queries

1. Find the First and Last Movie Released Each Year

Retrieve the first and last movie released in each year. Display the year, first movie title, and last movie title.

Code:

-- Question 1 --

```
SELECT strftime('%Y', m.release_date) AS year,
```

```
    -- First movie released in that year
```

```
(SELECT m1.title
```

```
FROM movies AS m1
```

```
WHERE strftime('%Y', m1.release_date) = strftime('%Y', m.release_date)
```

```
ORDER BY m1.release_date ASC
```

```
LIMIT 1) AS first_movie,
```

```
-- Last movie released in that year
```

```
(SELECT m2.title
```

```
FROM movies AS m2
```

```
WHERE strftime('%Y', m2.release_date) = strftime('%Y', m.release_date)
```

```
ORDER BY m2.release_date DESC
```

```
LIMIT 1) AS last_movie
```

```
FROM movies AS m
```

```
GROUP BY year
```

```
ORDER BY year;
```

Screenshot:

	year	first_movie	last_movie	
1	1916	Intolerance	Intolerance	
2	1925	The Big Parade	The Big Parade	
3	1927	Metropolis	Metropolis	
4	1929	Pandora's Box	The Broadway Melody	
5	1930	Hell's Angels	Hell's Angels	
6	1932	A Farewell to Arms	A Farewell to Arms	
7	1933	42nd Street	She Done Him Wrong	

Execution finished without errors.
Result: 90 rows returned in 204ms
At line 1:
-- Question 1: Retrieve the first and last movie released in each year. Display the year, first movie title, and last movie title. --
SELECT strftime('%Y', m.release_date) AS year,

-- First movie released in that year
(SELECT m1.title
FROM movies AS m1

2. Total Revenue Generated Each Year

Calculate the total revenue generated by movies released in each year. Display the year and total revenue, ordered by revenue in descending order.

Code:

-- Question 2 --

```
SELECT strftime('%Y', release_date) AS year, SUM(revenue) AS total_revenue
```

```
FROM movies
```

```
GROUP BY year
```

```
ORDER BY total_revenue DESC;
```

Screenshot:

	year	total_revenue
1	2012	24141710246
2	2014	24120490589
3	2013	23411493296
4	2015	22775024221
5	2009	21072651506
6	2011	20516921160
7	2010	20348574768

Execution finished without errors.
Result: 90 rows returned in 10ms
At line 22:
-- Question 2: Calculate the total revenue generated by movies released in each year. Display the year and total revenue, ordered by revenue in descending order. --
SELECT strftime('%Y', release_date) AS year, SUM(revenue) AS total_revenue
FROM movies
GROUP BY year
ORDER BY total_revenue DESC;

3. Movies Released in the First Half of the Year

Find all movies that were released between January and June. Display the movie title, release date, and director name.

Code:

-- Question 3 --

```
SELECT movies.title, movies.release_date, directors.name AS director_name
```

```
FROM movies
```

```
JOIN directors
```

```
ON movies.director_id = directors.id
```

```
WHERE strftime('%m', movies.release_date) BETWEEN '01' AND '06'
```

```
ORDER BY movies.release_date;
```

Screenshot:

	title	release_date	director_name
1	Metropolis	1927-01-10	Fritz Lang
2	Pandora's Box	1929-01-30	G.W. Pabst
3	The Broadway Melody	1929-02-08	Harry Beaumont
4	42nd Street	1933-02-02	Lloyd Bacon
5	She Done Him Wrong	1933-02-09	Lowell Sherman
6	It Happened One Night	1934-02-22	Frank Capra
7	Modern Times	1936-02-05	Charlie Chaplin

Execution finished without errors.
Result: 2166 rows returned in 10ms
At line 28:
-- Question 3: Find all movies that were released between January and June. Display the movie title, release date, and director name. --
SELECT movies.title, movies.release_date, directors.name AS director_name
FROM movies
JOIN directors
ON movies.director_id = directors.id
WHERE strftime('%m', movies.release_date) BETWEEN '01' AND '06'

4. Number of Movies Released on Weekends

Count the total number of movies released on a weekend (Saturday or Sunday). Use SQLite's strftime('%w') to determine the day of the week (0 = Sunday, 6 = Saturday).

Code:

```
-- Question 4 --
```

```
SELECT COUNT(*) AS weekend_movies
```

```
FROM movies
```

```
WHERE strftime('%w', release_date) IN ('0', '6');
```

Screenshot:

	weekend_movies
1	451

Execution finished without errors.
Result: 1 rows returned in 11ms
At line 36:
-- Question 4: Count the total number of movies released on a weekend (Saturday or Sunday). Use SQLite's strftime('%w') to determine the day of the week (0 = Sunday, 6 = Saturday). --
SELECT COUNT(*) AS weekend_movies
FROM movies
WHERE strftime('%w', release_date) IN ('0', '6');

CASE Statement Queries

5. Categorize Directors Based on Total Revenue

Classify directors based on the total revenue of their movies:

- High Revenue if total revenue > \$1 billion
- Moderate Revenue if total revenue is between \$500 million and \$1 billion
- Low Revenue if total revenue < \$500 million

Code:

-- Question 5: --

```
SELECT directors.name AS director_name, SUM(movies.revenue) AS total_revenue,

CASE WHEN SUM(movies.revenue) > 1000000000 THEN 'High Revenue'

WHEN SUM(movies.revenue) BETWEEN 500000000 AND 1000000000 THEN 'Moderate Revenue'

ELSE 'Low Revenue' END AS revenue_category

FROM directors

JOIN movies

ON directors.id = movies.director_id

GROUP BY directors.id, directors.name

ORDER BY total_revenue DESC;
```

Screenshot:

	director_name	total_revenue	revenue_category
1	Steven Spielberg	9147393164	High Revenue
2	Peter Jackson	6498642820	High Revenue
3	James Cameron	5883569439	High Revenue
4	Michael Bay	5832524638	High Revenue
5	Christopher Nolan	4227483234	High Revenue
6	Chris Columbus	3725631503	High Revenue
7	Robert Zemeckis	3590622002	High Revenue
8	George Lucas	3339113893	High Revenue

Execution finished without errors.
Result: 2349 rows returned in 12ms
At line 41:
-- Question 5: Classify directors based on the total revenue of their movies:
--- • High Revenue if total revenue > \$1 billion
--- • Moderate Revenue if total revenue is between \$500 million and \$1 billion
--- • Low Revenue if total revenue < \$500 million

6. Movies Based on Popularity and Revenue

Classify movies into the following categories based on popularity and revenue:

- Blockbuster if popularity > 100 and revenue > \$500 million
- Hit if popularity > 50 and revenue between \$100 million and \$500 million
- Average for all other movies

Code:

-- Question 6 --

SELECT title, popularity, revenue,

CASE WHEN popularity > 100 AND revenue > 500000000 THEN 'Blockbuster'

WHEN popularity > 50 AND revenue BETWEEN 100000000 AND 500000000 THEN 'Hit'

ELSE 'Average' END AS movie_category

FROM movies;

Screenshot:

	title	popularity	revenue	movie_category
1	Avatar	150	2787965087	Blockbuster
2	Pirates of the Caribbean: At World's End	139	961000000	Blockbuster
3	Spectre	107	880674609	Blockbuster
4	The Dark Knight Rises	112	1084939099	Blockbuster
5	John Carter	43	284139100	Average
6	Spider-Man 3	115	890871626	Blockbuster
7	Tangled	48	591794936	Average
8	Avengers: Age of Ultron	134	1406403694	Blockbuster

Execution finished without errors.
Result: 4773 rows returned in 14ms
At line 55:
-- Question 6: Classify movies into the following categories based on popularity and revenue:
--- • Blockbuster if popularity > 100 and revenue > \$500 million
--- • Hit if popularity > 50 and revenue between \$100 million and \$500 million
--- • Average for all other movies

7. Directors with Critically Acclaimed and Poorly Rated Movies

List directors and count how many of their movies fall into these categories:

- Critically Acclaimed if vote_average ≥ 8
- Poorly Rated if vote_average < 5

Code:

-- Question 7 --

```
SELECT directors.name AS director_name,  
  
COUNT(CASE WHEN movies.vote_average  $\geq$  8 THEN 1  
  
END) AS critically_acclaimed,  
  
COUNT(CASE WHEN movies.vote_average  $<$  5 THEN 1  
  
END) AS poorly Rated  
  
FROM directors  
  
JOIN movies  
  
ON directors.id = movies.director_id  
  
GROUP BY directors.id, directors.name
```

ORDER BY critically_acclaimed DESC;

Screenshot:

	director_name	critically_acclaimed	poorlyRated	
1	Christopher Nolan	5	0	
2	Peter Jackson	3	0	
3	Francis Ford Coppola	3	0	
4	Hayao Miyazaki	3	0	
5	Sergio Leone	3	0	
6	Robert Zemeckis	2	0	
7	Martin Scorsese	2	0	
8	David Fincher	2	0	

Execution finished without errors.
Result: 2349 rows returned in 11ms
At line 65:
-- Question 7: List directors and count how many of their movies fall into these categories:
--- • Critically Acclaimed if vote_average >= 8
--- • Poorly Rated if vote_average < 5

AI Usage: AI was used on question 1 and question 5. For question 1, AI was used to understand subqueries and how it works in SQLite. For question 5, AI was used to determine the syntax order when implementing joins and case statements.